شرح دورة عمل البروجكت بالكامل خطوة بخطوة

الفكرة العامة للبروجكت

البروجكت عبارة عن CPU بسيط بيشتغل بطريقة:

1. يجيب التعليمة من الذاكرة
2. يفهم نوع التعليمة
3. ينفذها باستخدام الـ Registers والـ BUS والـ ALU
4. يخزن النتيجة
5. يبدأ التعليمة اللي بعدها

كل ده بيحصل مع كل نبضة Clock.

المكونات الأساسية في البروجكت

1) PC — Program Counter

ده الريجستر اللي بيحتفظ بعنوان التعليمة الحالية.

مثال:

PC = 000

يعني أول تعليمة موجودة في Memory[000].

بعد كل Fetch:

PC ← PC + 1

عشان يجهز للتعليمة اللي بعدها.

---

2) AR — Address Register

بيمسك عنوان الذاكرة اللي هنقرأ أو نكتب منه.

لازم العنوان يعدي على Memory يعني أي عملية AR.

---

3) IR — Instruction Register

بيخزن التعليمة الحالية.

التعليمة بتنقسم إلى:

Opcode + Address

مثال:

ADD 200

يبقى:

Opcode = ADD
Address = 200

---

4) Decoder

ياخد الـ Opcode ويفهم نوع التعليمة.

مثال:

Opcode = 0010

Decoder يطلع:

INST_ADD = 1

وباقي التعليمات = 0.

---

5) SC — Sequence Counter

ده اللي بيطلع التوقيتات:

T0
T1
T2
T3
T4
T5
T6
T7

كل Clock ينقلنا لمرحلة جديدة.

---

6) BUS

الباص هو الطريق اللي البيانات بتمشي عليه.

أي Register عايز يبعت Data لازم يطلعها على BUS.

وأي Register عايز يحمل Data ياخدها من BUS.

خريطة الباص

000 → AR
001 → PC
010 → DR
011 → AC
100 → IR
101 → MEMORY
110 → INPR
111 → ZERO

مثال:

لو:

BUS_S = 010

يبقى:

BUS_OUT = DR_OUT

---

7) DR — Data Register

بيمسك البيانات اللي جاية من الذاكرة.

مثال:

DR ← Memory[AR]

---

8) AC — Accumulator

أهم Register في العمليات الحسابية والمنطقية.

غالبًا تشتغل على ALU الـ:

AC + حاجة
AC AND حاجة

والنتيجة ترجع AC.

---

9) ALU

مسؤولة عن العمليات الحسابية والمنطقية.

مثل:

ADD
AND
CMA
PASS

الـ ALU تاخد Inputs:

ALU_X = AC_OUT
ALU_Y = BUS_OUT

وتطلع:

ALU_OUT

---

10) MEMORY

الميموري تخزن:

- التعليمات
- البيانات

فيها:

Address
Data
Read
Write
Clock

---

11) OUTR — Output Register

الريجستر المسؤول عن الإخراج.

لما تنفذ:

OUT

القيمة تنتقل:

AC → OUTR

دورة عمل التعليمة بالكامل

المرحلة الأولى — Fetch

دي مرحلة جلب التعليمة من الذاكرة.

T0 — Fetch Address

عايزين:

AR ← PC

يعني:

1. يطلع العنوان على الباص الـ PC
2. يحمل العنوان AR

إشارات الكنترول:

BUS_S = 001
AR_LD = 1

الداتا باث:

PC_OUT → BUS → AR_IN

بعد الـ Clock:

AR ← PC

T1 — Fetch Instruction

عايزين:

IR ← Memory[AR]
PC ← PC + 1

الخطوات:

1. AR يحدد عنوان الذاكرة
2. Memory تطلع التعليمة
3. التعليمة تمشي على الباص
4. IR يحمل التعليمة
5. PC 1 يزيد

إشارات الكنترول:

M_READ = 1
BUS_S = 101
IR_LD = 1
PC_INC = 1

الداتا باث:

Memory → BUS → IR

بعد الـ Clock:

IR ← Memory[AR]
PC ← PC + 1

T2 — Decode

دلوقتي فيها IR الـ:

Opcode + Address

اللي يحصل:

IR_OPCODE → Decoder

والـ Decoder يطلع:

INST_ADD
INST_LDA
INST_AND
INST_STA
...

وفي نفس الوقت:

AR ← IR_ADDRESS

إشارات الكنترول:

BUS_S = 100
AR_LD = 1

T3 — Indirect Cycle (اختياري)

لو التعليمة Indirect:

I = 1

يبقى:

AR ← Memory[AR]

يعني نقرأ العنوان الحقيقي من الميموري.

Execute — مرحلة التنفيذ

دي أهم مرحلة.

الـ Execute Control يشوف:

Instruction + Timing

ويقرر:

مين يطلع على الباص
مين يتحمل
هل ALU تشتغل
هل Memory تقرأ
هل Memory تكتب

تنفيذ ADD

التعليمة:

ADD X

المطلوب:

AC ← AC + Memory[X]

T3

DR ← Memory[AR]

إشارات الكنترول:

M_READ = 1
BUS_S = 101
DR_LD = 1

T4

عايزين:

AC ← AC + DR

الخطوات:

1. DR يطلع على الباص
2. ALU تاخد:

AC_OUT
BUS_OUT

3. تعمل جمع
4. النتيجة ترجع AC

إشارات الكنترول:

BUS_S = 010
ALU_ADD = 1
AC_LD = 1
E_LD = 1
SC_CLR = 1

الداتا باث:

DR → BUS
BUS → ALU_Y
AC → ALU_X
ALU_OUT → AC

تنفيذ AND

AC ← AC AND DR

إشارات الكنترول:

ALU_AND = 1
AC_LD = 1

تنفيذ LDA

AC ← Memory[AR]

الخطوات:

Memory → DR
DR → AC

تنفيذ STA

Memory[AR] ← AC

الخطوات:

1. AC يطلع على الباص
2. الباص يروح للميموري
3. Memory تكتب

إشارات الكنترول:

BUS_S = 011
M_WRITE = 1
SC_CLR = 1

تنفيذ BUN

PC ← AR

يعني Jump.

تنفيذ ISZ

Memory[AR] ← Memory[AR] + 1

ولو النتيجة = 0:

PC ← PC + 1

تنفيذ OUT

OUTR ← AC

الخطوات:

1. AC يطلع على الباص
2. OUTR يحمل القيمة

إشارات الكنترول:

BUS_S = 011
OUTR_LD = 1

الداتا باث:

AC → BUS → OUTR

تنفيذ INP

AC ← INPR

إشارات الكنترول:

BUS_S = 110
AC_LD = 1

تنفيذ CMA

AC ← NOT AC

الـ ALU تعمل Complement.

تنفيذ CLA

AC ← 0

تنفيذ HLT

S ← 0

يعني وقف المعالج.

دور الـ Execute Control

الـ Execute Control هو مخ البروجكت.

باخد:

T0..T7
Instruction Signals
Flags

ويطلع:

AR_LD
DR_LD
AC_LD
IR_LD

PC_INC
PC_LD
M_READ
M_WRITE
BUS_S0
BUS_S1
BUS_S2
ALU_ADD
ALU_AND
ALU_PASS
ALU_CMA
SC_CLR

دور الـ SC_CLR

بعد ما التعليمة تخلص:

SC_CLR = 1

فيحصل:

SC ← 0

ونرجع:

T0

ونبدأ Fetch لتعليمة جديدة.

الدورة الكاملة المختصرة

فيه عنوان التعليمة PC
↓
AR ← PC
↓
تطلع التعليمة Memory

↓
IR ← Memory[AR]
↓
PC يزيد واحد
↓
IR_OPCODE يروح لل Decoder
↓
Decoder يطلع INST_ADD أو INST_LDA أو غيره
↓
IR_ADDRESS → AR
↓
Execute Control يشوف:
Instruction + Timing
↓
يطلع control signals
↓
BUS ينقل المصدر الصح
↓
Registers تتحمل
↓
ALU تنفذ لو التعليمة محتاجة ALU
↓
النتيجة ترجع AC أو Memory أو OUTR
↓
SC_CLR يرجعنا T0 للتعليمة اللي بعدها

مراحل البروجكت كاملة

T0 → Fetch Address
T1 → Fetch Instruction
T2 → Decode
T3 → Indirect (اختياري)
T4/T5/T6/T7 → Execute
SC_CLR → رجوع لـ T0